

Proyecto “Imperios en Guerra” — guía para exponer

RTS 1v1 estilo Age of Empires · **Unity 6000.6.0f1 + C#** · arquitectura **MVC** con **conurrencia real** ·
repo RTS-Academico · reparto: **Johan = Modelo + Controlador**, compañero = Vista.

1. Arquitectura MVC (la idea central)

Vista (Unity)		Controlador		Modelo (POCO, sin UnityEngine)
GestorJuego	—único→	JuegoControlador	—delega→	Simulacion (lock del mundo)
VistaTablero		· acciones		· 8 Tasks en background
HudRecursos		· parseo de red		· ConcurrentQueue de salida
ControlInputUsuario		· logs (vía cola)		· Instantanea() bajo lock
PanelFinPartida, MenuRed		· CERO Task/Thread/lock		· ConectorRed (Thread TCP)

Capa	Carpeta	Qué hace	Qué NO hace
Modelo	Assets/Scripts/Modelo	Estado, reglas, toda la concurrencia , IA, red física, logs	No usa UnityEngine
Controlador	Assets/Scripts/Controlador	Puente delgado: traduce Vista→Modelo y red→Modelo	No crea hilos
Vista	Assets/Scripts/Vista	Pinta 1 instantánea/frame y llama a la API	No muta el Modelo ni crea hilos

Flujo por frame (contrato):

1. Awake/Start → new JuegoControlador(...) **una sola vez**
2. Update → ProcesarMensajesRedPendientes() + Instantanea() → pintar
3. OnDestroy/Quit → Detener()

2. Clases por capa

Modelo (Assets/Scripts/Modelo/) — 14 archivos, sin Unity

Clase	Rol
Simulacion (~1700 líneas)	Motor del mundo: mapa, reglas, combate BFS, items, batalla, espejo de red (*Rival), toda la concurrencia , candado lock(Candado)
ConectorRed	TCP host/cliente: hilo de escucha, colas concurrentes, reconexión cada 1 s
GestorArchivos	Logs en disco con productor-consumidor (BlockingCollection + hilo escritor)
IAEnemiga	IA PVE en su propia Task: recolecta → Cuartel → Soldados
DatosDelJuego	Catálogo de costos/stats + fábricas (CrearUnidad , configs)
InstantaneaJuego	Foto inmutable del mundo (lo único que pinta la Vista)
Unidad	Vida, ataque/defensa, destino, estado, item equipado
Edificio	Vida, construcción (EnConstruccion / Operativo), entrenables
Recurso	Yacimiento (cantidad, Extraer)
Item	Yogur / Casco / Espada / Herramientas en el mapa
Jugador	5 recursos, unidades, edificios, bonuses, PuedePagar / Gastar
Mapa	Rejilla 30×30 , transitabilidad, casillas libres
Partida	Tiempo real (sin turnos) + ganador
Tipos	Enums: TipoUnidad , TipoEdificio , TipoRecurso , TipoItem , estados

Controlador — 1 archivo

Clase	Rol
JuegoControlador	API pública de la Vista (no romper). Acciones: MoverUnidad , ConstruirEdificio , EntrenarUnidad , MoverAAtacar , recolección, items... ·

Clase	Rol
	Red: HospedarRed , ConectarRed , ProcesarMensajesRedPendientes · Vista: Instantanea() , Detener() · PVE: IniciarIA / DetenerIA · Batalla: IniciarBatalla , métricas

Vista (Assets/Scripts/Vista/) — 7 clases

Clase	Rol
GestorJuego	Raíz: crea el Controlador, auto-bootstrap de UI, update = red + 1 foto/frame
VistaTablero	Rejilla, sprites, interpolación de movimiento, unidades apiladas en círculo
ControlInputUsuario	Clics + teclas: QWER entrenar, 1-4 construir, Alt+QWER selección, C//Esc, H/J/M red
HudRecursos	Barra: 5 recursos + tiempo + modo (VS MAQUINA / RED / LOCAL) + mensajes
PanelFinPartida	Modal victoria/derrota + Reintentar
MenuRed	Menú provisional Host/Cliente (tecla M)
SpriteFactory	Pixel-art 16×16 generado por código (arte temporal hasta issues #4–#9)

Pruebas

Test	Dónde	Qué verifica
PruebasBatch.TODO()	Assets/Pruebas/Editor	Sin Controlador no hay juego · PVE avanza sin red · PVP handshake
PruebaEnUnity	Assets/Pruebas/Juego	Integración en Play: mover, construir, batalla
Suites escritorio	Temp\opencode\ (solo lectura)	Modelo sin Unity: rts-pve-test 37 OK, rts-red-test 8 OK

3. Concurrency (lo que más te pueden preguntar)

Regla de oro

Controlador y Vista = 0 hilos. Toda concurrency vive en el Modelo (`Simulacion` , `ConectorRed` , `GestorArchivos` , `IAEnemiga`).

Inventario

Mecanismo	Dónde	Para qué
8 Tasks	<code>Simulacion</code> + <code>IAEnemiga</code>	Ver abajo
2 Threads	<code>ConectorRed</code> (escucha TCP), <code>GestorArchivos</code> (escribe logs)	I/O bloqueante fuera del hilo de juego
<code>lock(Candado)</code>	<code>Simulacion</code>	Único candado del mundo (unidades, edificios, recursos, items, partida)
<code>lock(_lockEnvio)</code>	<code>ConectorRed</code>	Escritura TCP segura
<code>ConcurrentQueue</code>	Colas de red entrada/salida	Productor-consumidor red
<code>BlockingCollection</code>	<code>GestorArchivos</code>	Productor-consumidor de logs a disco
<code>CancellationToken</code>	Reloj, sim, spawner, entrenos, obras, recolección, IA	Cancelación limpia en <code>Detener()</code>
<code>volatile</code>	<code>_detenido</code> , <code>BucleActivo</code> , <code>_activa</code> ...	Flags visibles entre hilos

Las 8 Tasks del Modelo

1. **Reloj** — +1 s cada 1000 ms (tiempo real, sin turnos)
2. **Bucle de simulación** — cada **100 ms**: avanza destinos (BFS 1 casilla/latido) + combate si `BucleActivo`
3. **Entrenamiento** — Task por trabajo; al terminar anuncia `ENTRENAR;`... por la cola de red
4. **Construcción** — Task por obra; al terminar edificio operativo
5. **Recolección** — Task por aldeano; ciclo cada `CicloRecoleccionMs`
6. **Spawner de items** — solo host; cada 15 s, máx. 8 items en mapa

- 7. **Expiración del Casco** — quita bonus a los 10 s
- 8. **IA enemiga** — cada 2 s decide (economía → Cuartel → Soldados); muta solo con métodos `*IA bajo lock(Candado)`

3 patrones clave (para defender)

- 1. **Nunca I/O dentro del candado del mundo**
Las Tasks solo **encolan** mensajes; el Controlador **drena en el hilo principal** (`ProcesarMensajesRedPendientes`) y ahí hace el socket. Igual con logs: solo se encola, un hilo aparte escribe a disco.
- 2. **Foto segura**
`Instantanea()` copia las listas **bajo lock(Candado)** ; la Vista itera **su copia** 1 vez por frame → sin “Collection was modified”.
- 3. **Excepciones no ocultas**
`IniciarTarea(Task, nombre)` observa con `ContinueWith(OnlyOnFaulted)` y las registra en el log.
Nada de `_ = MetodoAsync()` ciego.

Concurrencia “visible” en pantalla: modo batalla (`IniciarBatalla(n)`) — un solo hilo late dentro del candado (Nivel 1, sin carreras).

4. Requerimientos (checklist para la defensa)

#	Requerimiento	Estado
1	MVC obligatorio: Modelo = estado + concurrencia; Controlador = puente; Vista = dibuja y llama	✓
2	Prohibido Task/Thread/lock en Controlador y Vista	✓ solo en Modelo
3	Modelo POCO: sin <code>using UnityEngine</code> , testeable fuera de Unity	✓
4	No romper la API pública de <code>JuegoControlador</code>	✓ contrato en <code>copilot-instructions.md</code>
5	Estado a la Vista solo lectura + copia bajo <code>lock</code> si es “vivo”	✓ <code>Instantanea</code> , <code>ItemsVisibles</code>

#	Requerimiento	Estado
6	Red = extra/secundario	✓ implementada, priorizada como extra
7	Logs: configuracion.txt , log_partida.txt , resultado_final.txt	✓ GestorArchivos async
8	GitHub, rama modelo , límite 22-sep	✓
9	Tareas con CancellationToken , cancelables desde Detener()	✓
10	Comentar [Concurrencia] en cada punto	✓
11	Commits en español, tipo feat(modelo): ...	✓
12	No tocar suites de prueba de Temp\opencode	✓ (solo lectura)

5. Funcionalidades jugables

- **Mapa 30×30** · ~17 yacimientos (Oro, Madera, Comida, Hierro, Piedra)
- **Economía:** recolección en background (adyacente o viaje + autoinicio)
- **Construcción:** Casa, Cuartel, Torre, Centro Urbano (termina sola vía Task)
- **Entrenamiento:** Aldeano, Soldado, Arquero, Caballero (Task + cancelación)
- **Movimiento:** destino + **BFS 4 direcciones**, 1 casilla/100 ms, sin teletransporte; **unidades apilables** (solo edificios bloquean)
- **Combate:** ataque en rango + **aproximación** (clic → camina hasta pegar); daño $\max(0, atq-def)$
- **Items concurrentes:** Yogur (cura), Casco (+defensa temporal), Espada (+ataque), Herramientas (+5% recolección)
- **PVE:** IA económica + militar
- **Victoria:** sin Centro Urbano o sin unidades → FIN; ganador sincroniza copias
- **Red P2P TCP** (extra): puerto **5505**, protocolo ; — SALUDO , MOVER , ATACAR , CONSTRUIR , ENTRENAR , RECOLECTAR , ITEM , RECOGER_ITEM , FIN · espejo “*quien actúa, avisa*” · reconexión automática · menú **M** / teclas **H/J**

6. Una frase para cerrar la exposición

MVC estricto donde el Modelo es POCO dueño de **8 Tasks + candado del mundo + colas productor-consumidor** (red y disco fuera de los locks), el Controlador es una **API fachada sin concurrencia**, y la Vista solo consume una **instantánea por frame**; encima, RTS completo (economía, BFS, combate, items, PVE) más **red TCP espejo con reconexión**, todo verificado con pruebas batch y suites de escritorio.

7. Anexo: dependencia con la “API” (si preguntan)

La “**API**” del juego es `JuegoControlador` (no hay API web/externa). La Vista solo habla con ella; sin `Controlador` , `GestorJuego.Update` sale temprano y no hay foto ni acciones → **el juego no corre**.

	API actual (<code>JuegoControlador</code>)	API web/externa (hipotética)
Qué es	Clase C# en tu proceso	Servidor fuera de Unity
Cómo se habla	<code>ctrl1.MoverUnidad(...)</code> en memoria	HTTP/JSON (<code>POST /mover</code>)
Latencia	Instantánea	Miles de ms
Si “cae”	No cae: es tu código	Servidor caído = sin juego
Estado	En RAM (<code>Simulacion</code>)	A menudo en BD del servidor

En un RTS, lo típico sería API web solo para **cuentas/rankings**; la simulación en vivo se queda **local** (o P2P TCP, como aquí).